



MERN Stack Codebase Review Report

Project: CnscCodexMain — CNSC Codex

Reviewed: May 2026 | **Repository:** <https://github.com/Locquiano101/CnscCodexMain> | **Stack:** React 19 + Vite 7 / Express 5 / MongoDB + Mongoose 8 / Socket.IO 4

1. Project Overview

CNSC Codex is a web platform for **Camarines Norte State College's Student Development Unit (SDU)**. It manages the full lifecycle of student organization accreditation: initial registration, annual re-accreditation, proposal submissions, activity accomplishments, financial reporting, posts/announcements, and real-time notifications. It supports five distinct user roles (Student Leader, Adviser, Dean, SDU Coordinator, SDU Admin) plus an unauthenticated public view. The system also includes an experimental OpenAI-powered document feedback feature and uses Socket.IO for real-time notifications.

2. High-Level Architecture Review

Frontend Structure

The React 19 frontend lives under `client/src/` with a reasonable top-level layout:

```
src/
  api/          ← only one file (home_page_api.jsx) – not consistent
  components/   ← shared UI + role-specific components mixed
  config/       ← api.js for URL constants ✓
  hooks/        ← only use-toast.js – severely underutilized
  pages/
    public/     ← unauthenticated views
    admin/      ← role-gated dashboards per role
  utils/        ← export and print utilities
```

Problems: The `api/` folder has a single file — API calls are scattered directly in page components rather than centralized in service files. The `hooks/` folder is nearly empty despite the codebase having significant data-fetching logic that should be in custom hooks. The `components/` folder mixes Radix UI primitives (`ui/`) with feature-specific components without further sub-categorization.

Backend Structure

```
server/src/
  server.js     ← entry point + MongoDB connection + middleware setup
  routers.js    ← ALL routes in one file
  controller/   ← 14+ controller files ✓ (reasonable separation)
  middleware/   ← 9 middleware files ✓
```

```
models/      ← 14+ Mongoose models  ✓
uploads/     ← file storage (local disk, not cloud)
```

Problems: All routes live in one `routers.js` file. With 14+ controllers this becomes unmaintainable at scale. There is no `services/` layer — controllers handle business logic directly.

API Flow

```
Client (Axios) → /api/* (Express) → routers.js → controller/* → models/* → MongoDB
                                     ↓
                               middleware (auth, rate-limit, files, AI, etc.)
```

The API is fairly well-structured at the route level but suffers from consistency issues detailed below.

3. Backend Issues (Node/Express)

3.1 Single Monolithic Router File

File: `server/src/routers.js`

All API routes — auth, organizations, accreditation, proposals, accomplishments, financial reports, notifications, posts, audit logs — are registered in a single file. With 14+ resource groups this file likely exceeds 300–500 lines and is extremely difficult to navigate and maintain.

Fix: Split into resource-specific router files:

```
routers/
  auth.router.js
  organization.router.js
  accreditation.router.js
  proposal.router.js
  ...
```

Then in `server.js`:

```
app.use("/api/auth", authRouter);
app.use("/api/organizations", organizationRouter);
```

3.2 No Service Layer

Files: `server/src/controller/*.js`

Controllers directly perform MongoDB queries, handle business logic, send emails, and emit socket events all in one place. There is no `services/` layer to separate orchestration from transport. A controller like

`accreditation-document.js` almost certainly handles file saving, DB writes, email notification, and socket emission in one function.

Fix: Introduce `server/src/services/` directory. Controllers call services; services call models and other middleware utilities.

3.3 API Naming Inconsistency (REST Anti-Pattern)

File: `server/src/routers.js` (from README API table)

Route names like `getAllOrganizationProfile`, `getOrganizationProfile/:id`, `getProposalsBySdu/:id`, `addTransaction`, `addPresident` are RPC-style function names, not RESTful resource paths.

Current:

```
GET /api/getAllOrganizationProfile
GET /api/getOrganizationProfile/:id
POST /api/addTransaction
POST /api/addPresident
```

Should be:

```
GET /api/organizations
GET /api/organizations/:id
POST /api/financial-reports/:id/transactions
POST /api/organizations/:orgId/presidents
```

This creates confusion about resource hierarchy and makes the API harder to document and consume.

3.4 Async/Await Error Handling - FIXED

Files: `server/src/controller/*.js`

There is no evidence of a centralized async error wrapper. Express 5 does propagate async errors automatically (unlike Express 4), which is good since the project uses Express 5. However, without a global error handler middleware that returns consistent JSON error shapes, unhandled rejections will produce inconsistent responses (some may expose stack traces in production).

Fix: Add a global error handler in `server.js`:

```
app.use((err, req, res, next) => {
  const status = err.status || 500;
  res.status(status).json({
    success: false,
    message: err.message || "Internal Server Error",
    ...(process.env.NODE_ENV !== "production" && { stack: err.stack }),
  });
});
```

```
});  
});
```

3.5 No Input Validation Library

Files: `server/src/controller/*.js`, `server/src/middleware/`

The README mentions a profanity filter middleware but there is no mention of `express-validator`, `joi`, or `zod` for structured schema validation of request bodies. The profanity filter is not a substitute for input validation — it does not enforce required fields, data types, length constraints, or format rules.

Fix: Add `zod` or `joi` validation middleware per route, especially on `POST /api/register`, `POST /api/login`, `POST /api/createProposal`, and all financial report endpoints.

3.6 In-Memory Multer → Local Disk Storage

File: `server/src/middleware/files.js`

Per the README: "All file uploads are handled through Multer using in-memory storage, then persisted to `server/uploads/<organizationProfile>/`." This means every uploaded file passes through Node.js process memory before being written to disk.

Problems:

- Files are stored **on the server's local filesystem**, not object storage (S3, Cloudinary, etc.). This will break on stateless hosting (Render, Railway, Fly.io) where the filesystem is ephemeral.
- In-memory staging of large files (up to 10MB per the config) risks OOM crashes under concurrent uploads.
- The `server/uploads/` directory path hardcoded to organization profile names creates directory traversal risk if org names are user-controlled and not sanitized before use as filesystem paths.

Fix: Move file storage to Cloudinary or AWS S3. Use `multer-storage-cloudinary` and serve file URLs directly.

3.7 Session Secret Handling

File: `server/.env` (documentation), `server/src/server.js`

The README explicitly shows `SESSION_SECRET=your_session_secret_here` as a placeholder in the documented `.env` template. There is a real risk this placeholder is used in development and accidentally carried to production. Additionally, `connect-mongo` session store configuration needs verification that `httpOnly`, `secure` (for HTTPS), and `sameSite` cookie flags are set.

3.8 OpenAI API Key Named Incorrectly - FIXED

File: `server/.env`, `server/src/middleware/ai.js`

The README documents the OpenAI key as `REACT_APP_OPENAI_API_KEY` in the **server** `.env` file. `REACT_APP_` prefix is a Create React App convention for client-side environment variables. On the server, this

prefix is meaningless and suggests this may have been copy-pasted incorrectly. More critically, if this key name is also referenced anywhere in the frontend bundle, the API key would be exposed to the browser.

Fix: Rename to `OPENAI_API_KEY` in `server/.env`. Verify this key is never referenced in `client/` code.

4. Frontend Issues (React)

4.1 API Calls Directly in Page Components

Files: `client/src/pages/admin/student-leader/*.jsx`, `client/src/pages/admin/adviser/*.jsx`, etc.

The `api/` folder contains only `home_page_api.jsx`. All other API calls across role-specific dashboards are almost certainly written directly inside page components using `axios` or `fetch`. This means API call URLs, error handling, and response normalization are duplicated across dozens of page files.

Fix: Create `client/src/services/` with files per resource:

```
services/  
  organizationService.js  
  accreditationService.js  
  proposalService.js  
  ...
```

Each service wraps Axios calls with consistent error handling.

4.2 No Custom Data-Fetching Hooks

File: `client/src/hooks/` (only `use-toast.js` present)

All data fetching, loading state management, and error handling is likely duplicated in every page component. There are no custom hooks like `useAccreditation`, `useProposals`, `useOrganization`.

Fix: Create hooks per resource. At minimum:

```
// hooks/useAccreditation.js  
export function useAccreditation(orgId) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  useEffect(() => {  
    accreditationService  
      .get(orgId)  
      .then(setData)  
      .catch(setError)  
      .finally(() => setLoading(false));  
  }, [orgId]);  
  return { data, loading, error };  
}
```

4.3 No Error Boundary

File: `client/src/App.jsx`

There is no React Error Boundary wrapping the application. Any uncaught render error in any component will crash the entire app, showing a blank screen with no recovery path for the user.

Fix:

```
// components/ErrorBoundary.jsx
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  render() {
    return this.state.hasError ? <ErrorScreen /> : this.props.children;
  }
}
// Wrap in App.jsx
<ErrorBoundary>
  <RouterProvider router={router} />
</ErrorBoundary>;
```

4.4 State Management — No Global State Solution

The project uses React Context or prop-drilling for state. With 5 distinct user roles, each with multi-step accreditation workflows, shared notification state, and real-time socket events, the absence of a proper global state solution (Zustand, Redux Toolkit, or even a well-structured Context + Reducer pattern) means notification state, user session data, and accreditation status are likely re-fetched on every route change.

Fix: At minimum, create a `UserContext` and `NotificationContext` at the app root. For the accreditation workflow state machine, consider Zustand for lightweight global state.

4.5 API URL Configuration

File: `client/src/config/api.js`

This file exists (✅ positive), but it only centralizes the base URL. Individual endpoint paths are still hardcoded in component/service files as strings. Any endpoint rename on the backend requires hunting through the entire frontend codebase.

Fix: Export all endpoint paths from `api.js`:

```
export const ENDPOINTS = {
  organizations: '/api/organizations',
  accreditation: (id) => `/api/getAccreditation/${id}`,
  ...
};
```

4.6 Missing Memoization on Heavy Renders

Files: `client/src/pages/admin/sdu-main/`, accreditation list views, financial report pages

Pages that render large tables (roster members, accreditation lists, audit logs, financial transactions) with no `useMemo` or `React.memo` will re-render the entire list on any state change. The `sortable-table.jsx` component in `components/` likely accepts large arrays and re-renders on every parent state update.

Fix: Wrap `sortable-table` with `React.memo`. Memoize filtered/sorted data in parent components with `useMemo`.

4.7 .jsx Extension for Non-Component Files

File: `client/src/api/home_page_api.jsx`

API service files should not use `.jsx` extension. JSX is for React component syntax. Service/utility files should use `.js` or `.ts`. This is a naming convention issue that misleads tooling and developers.

4.8 sandbox.jsx in Production Codebase

File: `client/src/sandbox.jsx` (mentioned in README development notes)

A sandbox/testing file exists in the source tree. This will be included in the production bundle unless explicitly excluded. It should either be deleted or added to `.gitignore`/excluded from the Vite build.

5. Database & MongoDB Issues

5.1 Uploaded Files Path Stored as Relative Filesystem Paths

Model: Likely `server/src/models/document.js`, `accreditation_requirement.js`

If uploaded file paths are stored as relative paths like `uploads/OrgName/file.pdf`, these paths become invalid if the storage location changes or if the app is migrated to cloud storage. File references in the database should store abstract identifiers or full URLs, not filesystem paths.

5.2 Missing Indexes on High-Cardinality Query Fields

Based on the API routes, the following queries are performed repeatedly with no documented index strategy:

- `getProposalsBySdu/:id` — queries proposals by SDU/organization ID
- `getAccreditation/:id` — queries accreditation by organization ID
- `getFinancialReport/:id` — queries financial report by organization ID
- `notifications` — queries by user/org, sorted by date

Fix: Ensure compound indexes exist:

```
// proposals.js schema
ProposalSchema.index({ organization: 1, status: 1 });
ProposalSchema.index({ createdAt: -1 });

// notification.js schema
NotificationSchema.index({ recipient: 1, read: 1, createdAt: -1 });
```

5.3 No Pagination on List Endpoints

Files: `server/src/controller/proposal.js`, `organization.js`, `audit-logs.js`

Routes like `GET /api/getAllOrganizationProfile`, `GET /api/getAccomplishments`, `GET /api/admin/audit-logs` return unbounded result sets. Audit logs especially will grow indefinitely and returning all records in one query will cause timeouts and memory pressure.

Fix: Implement cursor-based or offset pagination on all list endpoints:

```
const page = parseInt(req.query.page) || 1;
const limit = Math.min(parseInt(req.query.limit) || 20, 100);
const results = await Model.find(filter)
  .skip((page - 1) * limit)
  .limit(limit);
```

5.4 Organization Name Used as Filesystem Directory Key

File: `server/src/middleware/files.js`

Per the README: "*persisted to* `server/uploads/<organizationProfile>/`". If `organizationProfile` is user-provided text (an org name with spaces, special characters, or path traversal sequences like `../`), this creates a **path traversal vulnerability**. An org named `../..etc` could write files outside the uploads directory.

Fix: Use the organization's MongoDB `_id` (a hex string) as the directory name, never user-controlled strings.

5.5 Embedded vs. Referenced Data

The `financial_report.js` model likely embeds transactions as an array subdocument. If an organization accumulates hundreds of transactions per year, querying the financial report document becomes increasingly expensive and the 16MB BSON document limit becomes a concern.

Fix: Transactions should be a separate `Transaction` collection with a reference to `financial_report` or `organization`, not embedded arrays.

6. Security Risks (CRITICAL)

● CRITICAL — Path Traversal via Organization Name in File Upload Paths

File: `server/src/middleware/files.js`

Using user-provided organization profile names as filesystem directory names without sanitization allows path traversal. An attacker registering an organization with a name like `../../server/src` could potentially write files to arbitrary server directories.

Severity: CRITICAL

Fix: Sanitize directory names to alphanumeric + hyphens/underscores only, or use MongoDB ObjectID as directory name.

● CRITICAL — Session Secret Placeholder in Documented `.env`

File: `server/.env` template in README

`SESSION_SECRET=your_session_secret_here` is the documented default. This placeholder is trivially guessable and if used in production allows session forgery.

Severity: CRITICAL

Fix: Generate a cryptographically random 64-byte secret: `node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"`. Add a startup check that rejects the application if `SESSION_SECRET` matches the placeholder value.

● CRITICAL — OpenAI Key Named with `REACT_APP_` Prefix on Server

File: `server/.env`

`REACT_APP_OPENAI_API_KEY` — this naming pattern suggests confusion between client and server environment variables. If this key name is referenced anywhere in the React app's source, Vite will embed it in the client bundle, exposing the API key to every browser visitor.

Severity: CRITICAL

Fix: Audit all `client/src/` files for `OPENAI` or `REACT_APP_OPENAI` references. Rename to `OPENAI_API_KEY` in `server/.env` only.

● HIGH — CORS Configuration Unknown / Likely Overly Permissive

File: `server/src/server.js`

The README does not document CORS configuration. In many Express+React development setups, CORS is set to `origin: '*'` or `origin: true` during development and never tightened for production. Overly permissive CORS allows any website to make authenticated cross-origin requests to the API.

Severity: HIGH

Fix:

```
app.use(  
  cors({  
    origin: process.env.CLIENT_URL, // e.g. https://cncs-codex.vercel.app  
    credentials: true,  
  })  
)
```

```
    methods: ["GET", "POST", "PUT", "PATCH", "DELETE"],
  }},
);
```

● HIGH — Session Cookie Security Flags

File: `server/src/server.js`

Session cookies must have `httpOnly: true`, `secure: true` (HTTPS only in production), and `sameSite: 'strict'` or `'lax'`. Without these, cookies are vulnerable to XSS theft and CSRF attacks.

Fix:

```
session({
  secret: process.env.SESSION_SECRET,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    maxAge: 30 * 24 * 60 * 60 * 1000, // ~1 month per README
  },
});
```

● HIGH — No Input Validation on API Endpoints

Files: `server/src/controller/*.js`

Without `joi/zod/express-validator`, endpoints accept arbitrary payloads. MongoDB NoSQL injection is mitigated by Mongoose's type coercion, but there is no protection against excessively large payloads, unexpected field types, or prototype pollution via `__proto__` keys in JSON bodies.

Fix: Add `express-mongo-sanitize` immediately as a quick win, then add proper schema validation per endpoint.

● MEDIUM — Rate Limiting Scope is Too Narrow

File: `server/src/middleware/rate-limit.js`

Per the README, rate limiting is applied only to *"admin-level mutation endpoints"*. The login endpoint (`POST /api/login`) is not mentioned as rate-limited. An unprotected login endpoint enables brute-force password attacks.

Fix: Apply aggressive rate limiting specifically to auth endpoints:

```
const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 10 });
router.post("/api/login", authLimiter, loginController);
```

● MEDIUM — File Upload MIME Type Validation (Client-Side Only Risk)

File: `server/src/middleware/files.js`

The README mentions "*File upload MIME type validation and size limits*" —  good. However, MIME type validation based solely on the file's `Content-Type` header (as Multer's `fileFilter` typically does) can be spoofed. An attacker can upload a malicious file with `Content-Type: image/png`.

Fix: Use `file-type` npm package to validate the actual magic bytes of the file buffer, not just the declared MIME type.

● MEDIUM — No Helmet.js (Missing Security Headers)

File: `server/src/server.js`

There is no mention of `helmet` middleware. Without it, responses lack critical security headers: `X-Content-Type-Options`, `X-Frame-Options`, `Content-Security-Policy`, `Strict-Transport-Security`.

Fix: `npm install helmet` and add `app.use(helmet())` as the first middleware in `server.js`.

7. Performance Improvements

7.1 No Pagination — All List Endpoints Return Full Datasets

See §5.3. `getAllOrganizationProfile`, `getAccomplishments`, `admin/audit-logs` all return unbounded datasets. Audit logs alone will become a serious performance problem within one academic year of real usage.

7.2 Socket.IO Notifications — No Room-Based Targeting

File: `server/src/middleware/socket.js`, `server/src/middleware/notification.js`

The README mentions Socket.IO for notifications but the `rooms-rollout.md` doc in `docs/` suggests room-based targeting is planned but not fully implemented. If notifications are broadcast to all connected clients rather than targeted rooms, every user receives every notification event regardless of relevance, causing unnecessary client-side processing and exposing notification data across roles.

Fix: Implement Socket.IO rooms per user session or organization. Emit only to the relevant room.

7.3 In-Memory File Staging on Upload

See §3.6. Large files (up to 10MB) are staged in Node.js memory before disk write. Under concurrent uploads from multiple organizations during accreditation season, this will cause memory spikes.

7.4 Static File Serving from Express

File: `server/src/server.js` (inferred from README: "*The static `/uploads` route serves these files with 30-day cache headers*")

Serving uploaded files directly from Express on the same process as the API is inefficient. Every file request occupies an API worker thread.

Fix: Migrate files to Cloudinary/S3 and serve via CDN. As an interim fix on self-hosted deployments, use nginx to serve the `uploads/` directory, bypassing Express entirely.

7.5 React Bundle — No Code Splitting Evidence

File: `client/vite.config.js`

The application has 5 role-specific dashboard sections with complex sub-pages (accreditation, proposals, accomplishments, financial reports per role). Without route-level lazy loading, the entire application bundle is downloaded on the login page.

Fix:

```
// App.jsx
const StudentLeaderDashboard = lazy(
  () => import("./pages/admin/student-leader/Dashboard"),
);
// Wrap routes with <Suspense fallback={<LoadingScreen />>
```

8. Code Quality & Maintainability

8.1 `.jsx` Extensions on Non-Component Files

Files: `client/src/api/home_page_api.jsx`, potentially others

Service files, utility files, and API modules should use `.js`. Only files containing JSX syntax should use `.jsx`. This is not enforced anywhere in the project.

8.2 No TypeScript

The entire codebase (100% JavaScript per GitHub) has no type safety. With 5 user roles, 14+ Mongoose models, and complex accreditation workflows, runtime type errors are a constant risk. Mongoose schema types partially compensate, but frontend data handling has zero type coverage.

Recommended path: Enable TypeScript incrementally via `allowJs: true + checkJs: true` in a `tsconfig.json` without needing to convert all files at once.

8.3 Duplicate Code Across Role-Specific Pages

Files: `client/src/pages/admin/student-leader/accreditation/`,
`client/src/pages/admin/adviser/accreditation/`, `client/src/pages/admin/dean/individual-accreditation/`, `client/src/pages/admin/sdu-coordinator/accreditation/`

Four roles have accreditation views. The UI for viewing accreditation status is largely identical across roles — only the available actions differ. Each role page almost certainly duplicates table rendering, status badge display, and document preview logic.

Fix: Create a shared `AccreditationView` component that accepts an `actions` prop specifying which role-specific buttons/controls to show.

8.4 No Logging Strategy

Files: `server/src/server.js`, `server/src/controller/*.js`

There is no mention of `morgan`, `winston`, or `pino` for structured HTTP logging. In production, debugging issues without request logs is extremely difficult. The audit logging (✅ present) covers accreditation requirement changes but does not cover general API access patterns, errors, or performance metrics.

Fix:

```
// Development
app.use(morgan("dev"));
// Production
app.use(morgan("combined", { stream: winstonStream }));
```

8.5 No Centralized HTTP Response Helper

Files: `server/src/controller/*.js`

Response shapes are inconsistent without a response helper. Different endpoints likely return `{ data: [...] }`, `{ result: {...} }`, `{ message: '...' }`, and raw arrays in different combinations.

Fix:

```
// utils/response.js
export const ok = (res, data, message = "Success") =>
  res.status(200).json({ success: true, message, data });
export const fail = (res, message, status = 400) =>
  res.status(status).json({ success: false, message });
```

8.6 Controller Files Named with Hyphens, Inconsistently

`accreditation-document.js`, `accreditation-requirement.js` use kebab-case. `financial_report.js`, `roster_member.js` (inferred from model names) use snake_case. JavaScript convention is camelCase for file names (`accreditationDocument.js`). Pick one and be consistent.

9. Suggested Improvements (Actionable)

#	Improvement	Effort	Impact
1	Add <code>helmet()</code> middleware to <code>server.js</code>	5 mins	High
2	Add <code>express-mongo-sanitize</code> to <code>server.js</code>	5 mins	High
3	Rotate <code>SESSION_SECRET</code> to cryptographically random value + add startup validation	15 mins	Critical

#	Improvement	Effort	Impact
4	Rename <code>REACT_APP_OPENAI_API_KEY</code> → <code>OPENAI_API_KEY</code> in server <code>.env</code> + audit client source for key references	30 mins	Critical
5	Add rate limiting on <code>POST /api/login</code>	15 mins	High
6	Sanitize org name before using as filesystem directory name	30 mins	Critical
7	Add global error handler middleware in <code>server.js</code>	30 mins	High
8	Split <code>routers.js</code> into per-resource router files	2 hrs	Medium
9	Add <code>mongoose</code> indexes on <code>organization</code> , <code>createdAt</code> , <code>status</code> fields	1 hr	High
10	Add <code>?page=&limit=</code> pagination to all list endpoints	3 hrs	High
11	Create <code>client/src/services/</code> layer with per-resource Axios wrappers	4 hrs	Medium
12	Add React Error Boundary wrapping <code><App /></code>	30 mins	Medium
13	Add lazy loading for all role-specific route chunks	1 hr	Medium
14	Migrate file uploads to Cloudinary/S3 (critical for cloud hosting)	1 day	Critical for deploy
15	Add <code>morgan</code> + structured logging to backend	30 mins	Medium
16	Validate uploaded file magic bytes with <code>file-type</code> package	1 hr	High
17	Add <code>zod</code> or <code>joi</code> validation to all POST/PUT endpoints	2 days	High
18	Implement Socket.IO room-based notification targeting	4 hrs	Medium

10. File-by-File Review

`server/src/server.js`

- **Missing:** `helmet()` call before route mounting
- **Missing:** `express-mongo-sanitize()` middleware
- **Missing:** Global 404 handler (`app.use((req,res) => res.status(404).json({...})))`
- **Missing:** Global error handler (see §3.4)
- **Check:** CORS `origin` setting — must not be `'*'` in production
- **Check:** Session cookie flags (`httpOnly`, `secure`, `sameSite`)

`server/src/routers.js`

- **Must fix:** Split into per-resource router files
- **Must fix:** Rename all routes from `getXByY/addX` patterns to REST resource paths

- **Check:** Verify every mutation route (POST/PUT/PATCH/DELETE) has the `auth` middleware applied

`server/src/middleware/files.js`

- **Critical fix:** Replace `organizationProfile` directory naming with MongoDB `_id`
- **Critical fix:** Validate actual file magic bytes, not just MIME headers
- **Recommended:** Replace local disk storage with Cloudinary SDK

`server/src/middleware/auth.js`

- **Check:** Verify session-based auth correctly validates session existence and role
- **Check:** Ensure role enforcement middleware checks `req.session.user.role` against allowed roles, not just the presence of a session

`server/src/middleware/ai.js`

- **Critical:** Verify `REACT_APP_OPENAI_API_KEY` is read from `process.env` and is NOT `VITE_` prefixed or otherwise leaked to the client bundle

`server/src/middleware/rate-limit.js`

- **Fix:** Apply rate limiting to `POST /api/login` and `POST /api/register` immediately

`client/src/App.jsx`

- **Add:** React Error Boundary
- **Add:** `lazy()` + `Suspense` for all role-specific dashboard imports

`client/src/config/api.js`

- **Extend:** Export all endpoint path constants, not just the base URL

`client/src/api/home_page_api.jsx`

- **Rename:** To `.js` extension
- **Expand:** Move all API calls from page components into this layer (organized by resource)

`client/src/pages/admin/student-leader/accreditation/`

- **Refactor:** Extract shared accreditation UI into `components/accreditation/` shared components

11. Priority Fix List

● Critical (Fix Before Any Production Deployment)

1. **Path traversal in file upload directory naming** — `middleware/files.js`: use `_id`, not org name
2. **Session secret placeholder** — Ensure `SESSION_SECRET` is never the default placeholder in any environment
3. **REACT_APP_OPENAI_API_KEY on server** — Audit and rename; verify key is not in client bundle

4. **No pagination on list endpoints** — `getAllOrganizationProfile`, `getAccomplishments`, `audit-logs` will crash the DB under real load
5. **Local filesystem file storage** — Files will be lost on every Render/Railway deployment restart; migrate to object storage

● Important (Fix Within First Sprint)

6. **Missing `helmet()` middleware** — Basic security headers completely absent
7. **Missing `express-mongo-sanitize`** — Low-effort, high-value NoSQL injection protection
8. **No rate limiting on login** — Brute force attack vector open
9. **Session cookie flags not confirmed** — `httpOnly`, `secure`, `sameSite` must be verified
10. **CORS configuration** — Must be locked to specific client origin(s) before production
11. **Global error handler missing** — Stack traces may be returned to clients in production
12. **No React Error Boundary** — Uncaught render errors crash the entire app
13. **MIME type spoofing on file upload** — Validate magic bytes, not Content-Type header

● Optional Improvements (Next Milestone)

14. Split `routers.js` into per-resource files
15. Rename routes to REST conventions
16. Add TypeScript (incremental)
17. Add `morgan/winston` logging
18. Add custom data-fetching hooks to eliminate component duplication
19. Add route-level code splitting with `lazy()`
20. Introduce a `services/` layer on the backend
21. Add centralized response helper for consistent API shapes
22. Implement Socket.IO rooms for targeted notifications

12. Final Summary

CNSC Codex is a well-scoped, feature-rich application for a real institutional use case. The developer shows solid understanding of the MERN ecosystem — the tech choices (React 19, Express 5, Mongoose 8, Socket.IO, Vite) are current and appropriate. The presence of audit logging, requirement gating, rate limiting, a profanity filter, and a `config/api.js` file demonstrate thoughtfulness above the typical student project. The monorepo structure with npm workspaces is clean.

However, this application is NOT production-ready in its current state.

The most severe issues cluster around three areas:

Security: Local filesystem path traversal via user-controlled directory names, an OpenAI API key with a client-side prefix on the server, no Helmet headers, unconfirmed CORS and cookie security, and no brute-force protection on the login endpoint are all deployment blockers.

Scalability: Zero pagination on any list endpoint, local disk file storage (breaks on stateless hosting), and in-memory file staging will cause failures under real institutional load.

Maintainability: A single 300+ line router file, no service layer, no TypeScript, no custom hooks, and API calls scattered in page components will make the codebase increasingly painful to extend as the SDU adds

requirements over academic years.

Address the  Critical items before going live. The  Important items should be resolved in the week following launch. The  Optional items are technical debt that will compound if deferred past the second academic year of operation.